

1. Cơ sở lý thuyết về Retrieval-Augmented Generation

1.1. Hạn chế của LLM thuần

- Các mô hình ngôn ngữ lớn LLM (Large Language Model) như GPT (OpenAI), Llama (Meta), Gemini (Google), Claude (Anthropic) hay các mô hình tương tự được huấn luyện trên một khối lượng dữ liệu văn bản khổng lồ, thu thập dữ liệu từ nhiều nguồn khác nhau trên Internet tính đến một thời điểm nhất định. Tri thức mà mô hình có được hoàn toàn nằm trong các tham số (parameters) đã học được trong quá trình huấn luyện, và tri thức này bị “đóng băng” lại tại thời điểm dữ liệu huấn luyện được thu thập.
- Điều này dẫn đến hai hạn chế khi áp dụng LLM vào các bài toán như tư vấn chuyên biệt, cụ thể như hệ thống mà nhóm đang triển khai là tư vấn ngành học tại Trường Đại học Công Thương TP.HCM:
 - Thứ nhất, LLM không có tri thức riêng về các thông tin đặc thù của một tổ chức cụ thể - ví dụ như chương trình đào tạo, tổ hợp xét tuyển. hay các thông tin cập nhật theo từng năm tuyển sinh của trường. Đây là những thông tin mà một LLM thuần – tức mô hình chỉ dựa vào tri thức đã học từ dữ liệu huấn luyện và không có khả năng truy cập nguồn dữ liệu ngoài – hầu như không thể trả lời chính xác, do thông tin hoặc không xuất hiện trong dữ liệu huấn luyện gốc, hoặc đã lỗi thời tại thời điểm mô hình được huấn luyện. Mặc dù một số LLM hiện nay được tích hợp thêm khả năng tìm kiếm web (web search) để bù đắp hạn chế này, cơ chế đó vẫn khác biệt với RAG ở chỗ nguồn thông tin không được kiểm soát và xác thực trước – kết quả tìm kiếm trên Internet có thể không chính xác, không đầy đủ hoặc không phản ánh đúng thông tin chính thức của trường, trong khi RAG cho phép hệ thống truy hồi trực tiếp từ một kho tri thức đã được xác thực và kiểm soát chất lượng từ trước.
 - Thứ hai, khi được hỏi về một vấn đề mà mô hình không thực sự nắm rõ, LLM có xu hướng tạo ra câu hỏi trả lời nghe có vẻ hợp lý, mạch lạc, nhưng lại có thể sai lệch so với thực tế. Hiện tượng này được gọi là ảo giác (hallucination) – một trong những vấn đề nghiêm trọng nhất khi

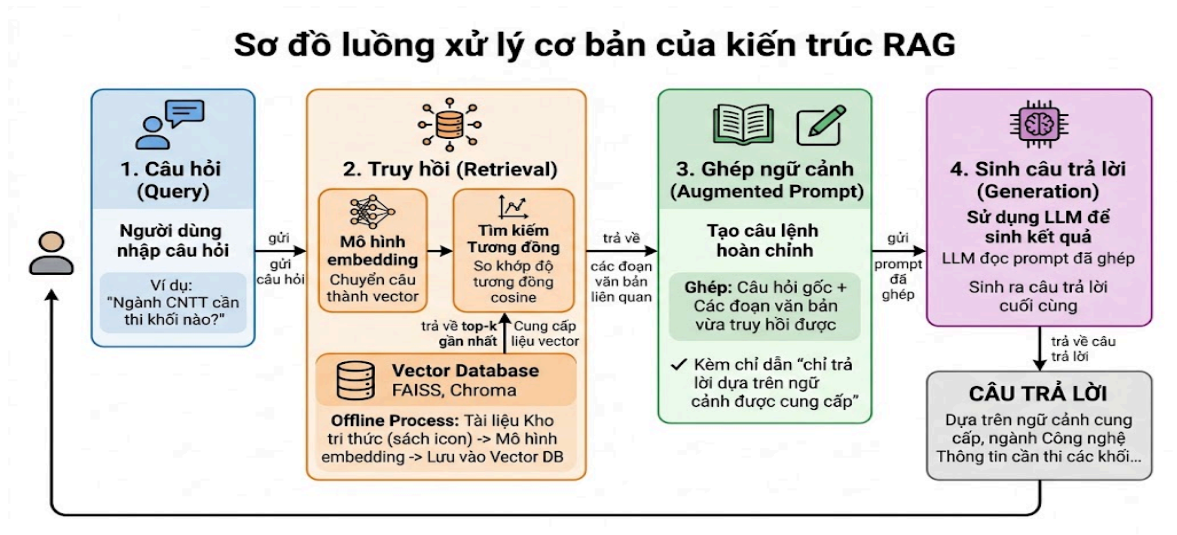
triển khai chatbot dựa trên LLM vào các hệ thống đòi hỏi độ chính xác tương đối cao. Đối với một hệ thống tư vấn tuyển sinh, nếu chatbot đưa ra thông tin sai về điều kiện xét tuyển hay một số nội dung chương trình học, hậu quả gây ra cho người dùng (học sinh, phụ huynh) là không nhỏ, ảnh hưởng trực tiếp đến quyết định chọn ngành học của họ.

- Chính hai hạn chế này là động lực để Lewis và cộng sự (2020) đề xuất kiến trúc Retrieval – Augmented Generation, với mục tiêu kết hợp khả năng sinh ngôn ngữ tự nhiên của LLM với khả năng truy xuất thông tin chính xác, cập nhật từ một nguồn tri thức bên ngoài.

1.2. Kiến trúc RAG

- RAG là một kiến trúc kết hợp giữ hai thành phần: một bộ truy hồi (retriever) và một bộ sinh văn bản (generator), hoạt động phối hợp với nhau thay vì để LLM trả lời hoàn toàn dựa vào tri thức có sẵn được học từ trước và không có sự bổ sung từ nguồn dữ liệu bên ngoài.
- Có thể hình dung cơ chế này tương tự như việc con người làm bài thi mở sách (open – book exam): thay vì buộc phải nhớ chính xác toàn bộ kiến thức, người làm bài được phép tra cứu tài liệu liên quan trước khi trả lời, từ đó đảm bảo câu trả lời có căn cứ thay vì chỉ dựa vào trí nhớ có thể hoặc không hoàn toàn chính xác.
- Quy trình xử lý một câu hỏi trong hệ thống RAG diễn ra theo bốn bước tuần tự:
 - Đầu tiên, hệ thống nhận câu hỏi (query) từ người dùng. Tiếp theo, bộ truy hồi (retriever) sẽ tìm kiếm trong kho tri thức đã chuẩn bị sẵn để lấy ra những đoạn văn bản có nội dung liên quan nhất đến câu hỏi đó. Sau khi có được các đoạn văn bản liên quan, hệ thống ghép chúng cùng với câu hỏi gốc thành một đoạn văn bản hoàn chỉnh, gọi là augmented prompt (câu lệnh đã được bổ sung ngữ cảnh). Cuối cùng, bộ sinh văn bản (generator), vốn là một LLM sẽ đọc toàn bộ augmented prompt này và sinh ra câu trả lời dựa trên ngữ cảnh vừa được cung cấp thay vì chỉ dựa vào tri thức đã học từ trước.
 - Nhờ cơ chế này, câu trả lời cuối cùng không còn hoàn toàn phụ thuộc vào những gì mô hình đã ghi nhớ lúc huấn luyện, mà được neo vào

những tài liệu thật, cụ thể, có thể kiểm chứng- đây chính là điểm cốt lõi giúp RAG giảm thiểu đáng kể hiện tượng ảo giác (hallucination) so với việc sử dụng LLM thuần.



1.3. Embedding và vector database

- Để bộ truy hồi có thể hiểu và so sánh được mức độ liên quan giữa các câu hỏi của người dùng với hàng trăm, hàng nghìn đoạn văn bản trong kho tri thức, toàn bộ văn bản cần được chuyển đổi sang một dạng biểu diễn mà máy tính có thể so sánh được về mặt ngữ nghĩa. Đây chính là vai trò của embedding.
- Embedding là quá trình chuyển đổi một đoạn văn bản thành một vector số học nhiều chiều (thường từ vài trăm đến hơn một nghìn chiều), được thực hiện bởi một mô hình chuyên biệt gọi là embedding model, **được biểu diễn bởi công thức (1)**. Đặc điểm quan trọng nhất của embedding là: hai đoạn văn bản có ý nghĩa càng gần nhau thì vector biểu diễn của chúng càng nằm gần nhau trong không gian nhiều chiều đó, và độ gần này được đo bằng một phép toán gọi là độ tương đồng cosine (cosine similarity), **được biểu diễn bởi công thức (2)**. Cosine similarity được lựa chọn thay vì khoảng cách Euclidean, **được biểu diễn bởi công thức (3)**, vì cosine similarity chỉ quan tâm đến hướng của vector (ý nghĩa ngữ nghĩa), không bị ảnh hưởng bởi độ lớn — trong khi Euclidean lại nhạy cảm với độ lớn này, nên kém phù hợp hơn cho bài toán so khớp ngữ nghĩa văn bản.

Công thức biểu diễn Embedding:

$$v_d = E(d) \quad (1)$$

Trong đó:

- d: đoạn văn bản đầu vào (một câu, một đoạn trong kho tri thức)
- E: embedding model (mô hình chuyển văn bản thành vector)
- v_d : vector kết quả, thuộc không gian n chiều ($v_d \in R^n$)

Ý nghĩa: Công thức này chỉ đơn giản "đưa văn bản d vào mô hình E, output ra vector v_d ". Không có gì phức tạp, chỉ là cách viết chuẩn hóa của bước "chuyển văn bản thành số".

Công thức biểu diễn độ tương đồng cosine:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

Trong đó:

- A, B: hai vector cần so sánh (ví dụ: vector câu hỏi và vector văn bản trong kho tri thức)
- $A \cdot B$: tích vô hướng (dot product) giữa A và B = tổng các cặp phần tử tương ứng nhân với nhau ($\sum A_i B_i$)
- $\|A\|$, $\|B\|$: độ dài (norm) của vector A, B = căn bậc hai tổng bình phương các phần tử ($\sqrt{\sum A_i^2}$)

Ý nghĩa: Cosine similarity đo mức độ "giống hướng" giữa hai vector, không quan tâm độ dài. Hai vector cùng hướng cho sim gần 1 (rất liên quan); vuông góc cho sim gần 0 (không liên quan); ngược hướng cho sim gần -1 (đối lập). Nhờ vậy, câu hỏi và văn bản dù dùng từ ngữ khác nhau nhưng cùng ý nghĩa vẫn cho ra sim cao, giúp hệ thống nhận diện được sự liên quan về ngữ nghĩa chứ không phải khớp từ ngữ.

Công thức khoảng cách Euclidean (đối chiếu, giải thích vì sao chọn cosine):

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2} \quad (3)$$

Trong đó:

- A, B: hai vector cần so sánh

- A_i, B_i : giá trị phần tử thứ i của vector A, B
- n : số chiều của vector

Ý nghĩa: Đây là khoảng cách đường thẳng giữa hai điểm trong không gian nhiều chiều, tương tự định lý Pytago mở rộng. Khác cosine ở chỗ nó tính cả độ lớn lẫn hướng của vector. Hai vector cùng hướng nhưng độ dài khác nhau vẫn bị tính là cách xa nhau theo Euclidean, dù về ngữ nghĩa có thể rất giống nhau. Đó là lý do Euclidean ít phù hợp hơn cosine cho bài toán so khớp văn bản.

- Ví dụ, nếu trong kho tri thức có đoạn văn bản mô tả "Ngành Công nghệ Thông tin xét tuyển theo các tổ hợp A00, A01, D01", và người dùng đặt câu hỏi "Muốn học CNTT thì cần thi khối nào?", thì mặc dù cách diễn đạt của hai câu này khác nhau hoàn toàn về mặt từ ngữ, embedding của chúng vẫn sẽ nằm gần nhau trong không gian vector, vì chúng mang ý nghĩa tương đồng.
- Sau khi toàn bộ dữ liệu trong kho tri thức đã được chuyển thành vector, các vector này cần được lưu trữ ở một nơi cho phép tìm kiếm nhanh chóng khi có câu hỏi mới – đây chính là vai trò của vector database (cơ sở dữ liệu vector). Khác với database được thiết kế chuyên biệt để thực hiện phép tìm kiếm tương đồng (similarity search) trên số lượng lớn vector một cách hiệu quả. Một số công cụ phổ biến cho mục đích này là FAISS và Chroma, việc lựa chọn và so sánh cụ thể giữa các công cụ này sẽ được trình bày chi tiết ở phần phương pháp nghiên cứu.

1.4. Kỹ thuật truy hồi và prompt augmentation

- Khi có một câu hỏi mới từ người dùng, quá trình truy hồi (retrieval) diễn ra như sau: câu hỏi được chuyển thành vector bằng một embedding model đã dùng để xử lý kho tri thức, sau đó vector này được so sánh với toàn bộ vector đã được lưu trong vector database để tìm ra một số lượng nhất định (thường kí hiệu là top- k , ví dụ như top-3 hoặc top-5) các đoạn văn bản có độ tương đồng cao nhất, **được biểu diễn bởi công thức (4).**

Công thức truy vấn top- k :

$$Top - k(q) = \operatorname{argsim}\left(v_q, v_{d_i}\right) \quad (4)$$

Trong đó:

- q : câu hỏi của người dùng
- v_q : vector của câu hỏi (tính theo công thức 1, $v_q = E(q)$)
- $D = \{d_1, d_2, \dots, d_N\}$: toàn bộ kho tri thức
- $\text{sim}(v_q, v_{d_i})$: độ tương đồng cosine giữa vector câu hỏi và vector văn bản d_i (tính theo công thức 2)
- argk max : phép chọn ra k phần tử d_i có giá trị sim lớn nhất

Ý nghĩa: Duyệt qua toàn bộ N văn bản trong kho tri thức, tính điểm sim giữa câu hỏi và từng văn bản, rồi chọn ra k văn bản có điểm cao nhất. Đây là bước retriever thực hiện việc "tìm tài liệu liên quan nhất" trước khi đưa vào LLM.

- Việc lựa chọn giá trị k phù hợp là một yếu tố quan trọng ảnh hưởng đến chất lượng câu trả lời: nếu k quá nhỏ, hệ thống sẽ có nguy cơ bỏ sót thông tin cần thiết; nếu k quá lớn, ngữ cảnh đưa vào mô hình sẽ chứa nhiều thông tin không liên quan, gây nhiễu và làm giảm chất lượng câu trả lời sinh ra cho câu hỏi.
- Sau khi đã có được các đoạn văn bản liên quan nhất, bước tiếp theo là prompt augmentation – quá trình xây dựng một câu lệnh hoàn chỉnh gửi đến LLM, bao gồm: câu hỏi gốc của người dùng, các đoạn văn bản vừa truy hồi được (đóng vai trò làm ngữ cảnh tham khảo), và thường kèm theo một số chỉ dẫn rõ ràng yêu cầu mô hình chỉ được trả lời dựa trên ngữ cảnh đã được cung cấp, không được tự suy diễn hay bổ sung thông tin ngoài phạm vi đó, **được biểu diễn bởi công thức (5)**. Cách thiết kế câu lệnh này (prompt engineering) ảnh hưởng trực tiếp đến việc mô hình có tuân thủ đúng ngữ cảnh được cung cấp hay không, và sẽ được trình bày cụ thể hơn trong phần phương pháp thực hiện.

Công thức Augmented prompt:

$$P_{aug} = \text{Concat}(I, q, d_1, d_2, \dots, d_k) \quad (5)$$

Trong đó:

- I : chỉ dẫn hệ thống (system instruction), ví dụ yêu cầu mô hình chỉ trả lời dựa trên ngữ cảnh cung cấp
- q : câu hỏi gốc của người dùng
- $d_1, \dots, d_k$: k văn bản vừa truy hồi được từ công thức (4)

- *Concat*: phép ghép chuỗi văn bản lại thành một khối duy nhất

Ý nghĩa: Không phải phép toán số học, chỉ là "dán" 3 phần lại theo thứ tự: hướng dẫn, câu hỏi, tài liệu tham khảo, tạo thành một prompt hoàn chỉnh gửi vào LLM.

1.5. Generation và đánh giá chất lượng câu trả lời

- Sau khi augmented prompt đã được xây dựng hoàn chỉnh, bước cuối cùng trong pipeline RAG là generation – quá trình LLM đọc toàn bộ prompt (bao gồm câu hỏi và ngữ cảnh) và sinh ra câu trả lời bằng ngôn ngữ tự nhiên, **được biểu diễn bởi công thức (6)**. Khác với việc sinh ra câu trả lời hoàn toàn tự do như khi sử dụng LLM thuần, ở bước này mô hình được định hướng để bám sát vào ngữ cảnh đã được cung cấp, từ đó tạo ra câu trả lời vừa tự nhiên, mạch lạc, vừa có căn cứ thực tế và có thể trích dẫn nguồn thông tin cụ thể.

Công thức sinh câu trả lời:

$$y = P(P_{aug}) \quad (6)$$

Trong đó:

- y' : một câu trả lời khả dĩ (ứng viên)
- $P(P_{aug})$: xác suất mô hình sinh ra câu trả lời y' khi biết trước prompt P_{aug}
- y : câu trả lời cuối cùng được chọn

Ý nghĩa: Trong vô số câu trả lời có thể sinh ra, mô hình chọn câu có xác suất cao nhất dựa trên prompt vừa đọc. Nói đơn giản là "câu trả lời hợp lý nhất dựa trên ngữ cảnh vừa cung cấp".

- Cần lưu ý rằng cơ chế RAG giúp đáng kể hiện tượng bị ảo giác nhưng không loại bỏ hoàn toàn nguy cơ này. Chất lượng câu trả lời cuối cùng phụ thuộc vào cả hai thành phần trong kiến trúc: nếu bộ truy hồi tìm sai hoặc không tìm đủ thông tin liên quan (lỗi ở tầng retrieval), hoặc nếu mô hình sinh vẫn tự thêm thông tin ngoài ngữ cảnh được cung cấp (lỗi ở tầng generation), câu trả lời cuối cùng vẫn có thể chứa sai sót. Vì vậy, việc đánh giá một hệ thống RAG cần xem xét đồng thời hai khía cạnh: độ chính xác của việc truy hồi (retrieval accuracy – hệ thống có tìm đúng tài liệu liên quan hay không), **đo bằng công thức (7), (8)**, và độ trung thực của câu trả lời sinh ra (faithfulness – câu trả lời có bám sát ngữ cảnh được cung cấp hay không, có trích dẫn đúng nguồn hay không). Các tiêu

chỉ đánh giá cụ thể cho hai khía cạnh này sẽ được trình bày ở phần phương pháp nghiên cứu của đề tài.

Công thức biểu diễn Precision@k:

$$Precision\ k = \frac{\text{số văn bản liên quan trong top-k}}{k} \quad (7)$$

Ý nghĩa: Trong k văn bản hệ thống trả về, bao nhiêu phần trăm thật sự liên quan. Ví dụ k=5, có 4/5 văn bản đúng thì Precision@5 = 0.8. Đo độ "sạch" của kết quả.

Công thức biểu diễn Recall@k:

$$Recall\ k = \frac{\text{số văn bản liên quan trong top-k}}{\text{tổng số văn bản liên quan có trong kho tri thức}} \quad (8)$$

Ý nghĩa: Trong toàn bộ văn bản đúng có sẵn trong kho tri thức, hệ thống tìm được bao nhiêu phần trăm. Ví dụ kho có 6 văn bản liên quan, hệ thống chỉ tìm ra 4 thì Recall = 0.67. Đo độ "đầy đủ", ngược lại với Precision.

Lưu ý chung cho (7) và (8): Precision cao mà Recall thấp nghĩa là hệ thống kỹ tính nhưng bỏ sót thông tin; Recall cao mà Precision thấp nghĩa là hệ thống lấy dư thông tin gây nhiễu. Đây là lý do phần 1.4 có nhắc đến việc chọn k phù hợp — hai chỉ số này thường đánh đổi lẫn nhau khi k thay đổi.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [2] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019. [Online]. Available: <https://arxiv.org/pdf/1702.08734>
- [3] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvassy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[Online]. Available: <https://arxiv.org/pdf/2401.08281>

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410.pdf>

[5] Chroma, "Introduction," *Chroma Documentation*. [Online]. Available: <https://docs.trychroma.com/docs/overview/introduction>. [Accessed: 17-Aug-2026].

[6] Học Lập Trình cùng Phát, "Retrieval-Augmented Generation (RAG) cơ bản cho các Mô Hình Ngôn Ngữ lớn (LLMs)," *YouTube*, Sep. 25, 2024. [Online]. Available: https://www.youtube.com/watch?v=Nb3_rq5Xnic